Implement client script and UI policy (Incident)

Phase-4 -- Project Planning Phase

Date	29.09.2026
Team Id	SWTID-2026-8014
Project Name	Implement client script and UI policy (Incident
Project Guide Name	Mrs.S.Raja Shyamala , M.Sc., M.Phil., B.Ed., TET

Project Objectives Planning

Define clear objectives to guide the implementation and verification steps:

- Import employee data from a spreadsheet into ServiceNow reliably.
- Use an Import Set as a staging area (u_employee_import).
- Use a Transform Map to map and move data into the Employee Test table (u_employee_test).
- Maintain Employee ID as the primary identifier and Coalesce key to prevent duplicates.
- Support both insert (new records) and update (existing records) operations.
- Validate transformed records and produce three reports and a centralized dashboard.

- Prepare the spreadsheet data before loading it into the ServiceNow Import Set.
- Create the **Employee Import** staging table to receive spreadsheet records.
- Create the **Employee Test** target table to store transformed employee records.
- Configure the Transform Map between the Employee Import and Employee Test tables.
- Map all five employee fields correctly during the transformation process.
- Use **Auto Map Matching Fields** where the source and target field names match.
- Use **Mapping Assist** for fields that require manual mapping.
- Test the transformation using new and existing Employee IDs.
- Verify the transformation results through **Transform History** and the Employee Test table.
- Provide employee data visibility through department, location, and employee-list reports on the **Employee Analytics Dashboards** dashboard.

Data Planning

Plan the source and target data structure and preparation rules before importing.

- Required fields in source and target: Employee ID, Employee Name, Email, Department, Location.
- Use consistent field names and formats across spreadsheet and ServiceNow tables.
- Place data in Sheet 1 with a header row (row 1) describing each column.
- Ensure Employee ID is populated for every row to enable Coalesce matching.
- Create a sample file named Sample Spreadsheet.xlsx for iterative testing.



- Define the **Employee ID** as the key employee identification field.
- Prepare employee data in a structured spreadsheet format before importing.
- Ensure each spreadsheet column corresponds to a field in the Employee Import and Employee Test tables.
- Keep **Employee Name** values ready for insertion or update during transformation.
- Prepare valid **Email** values for each employee record.
- Include **Department** information for department-wise reporting.
- Include **Location** information for location-wise reporting.
- Plan the data structure so the same employee can be identified during repeated imports.
- Prepare both **new employee records and existing employee records with changed values** for testing.
- Use the prepared sample data to verify the import, transformation, update, and duplicate- prevention process.

Table and Staging Planning

Define the staging and target tables, their purpose, and the rationale for separation.

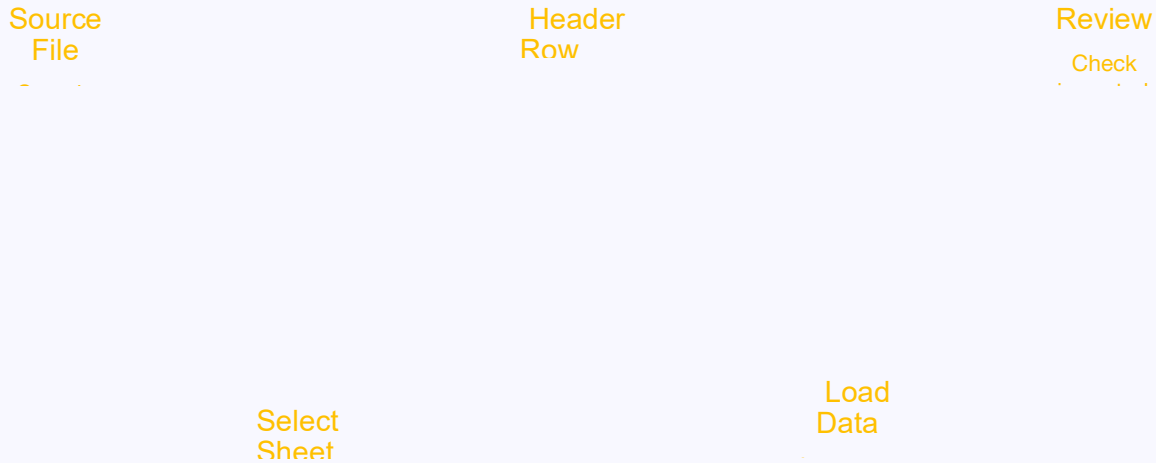
- Employee Import (staging): Label = Employee Import, Table = u_employee_import — receives raw spreadsheet rows.
- Employee Test (target): Label = Employee Test, Table = u_employee_test — final store for transformed, validated records.
- Keep staging and target separate to allow review and corrective action prior to transformation.
- Maintain identical field set between staging and target to simplify mapping.

- Define **Employee Import** as the staging table for uploaded spreadsheet data.
- Define **Employee Test** as the target table for transformed employee records.
- Use u_employee_import as the source table for the Transform Map.
- Use u_employee_test as the target table for the Transform Map.
- Plan the staging table to hold the imported spreadsheet rows before transformation.
- Plan the target table to contain the resulting employee records after transformation.
- Create the target table with fields for **Employee ID, Employee Name, Email, Department, and Location**.
- Verify that the required fields are available in the target table before transformation.
- Plan the table structure so source fields can be mapped to corresponding target fields.
- Verify the target Employee Test records after transformation to confirm that the planned table structure works correctly.

Import Set Planning

Plan the Import Set configuration and standard operating steps for loading spreadsheet data.

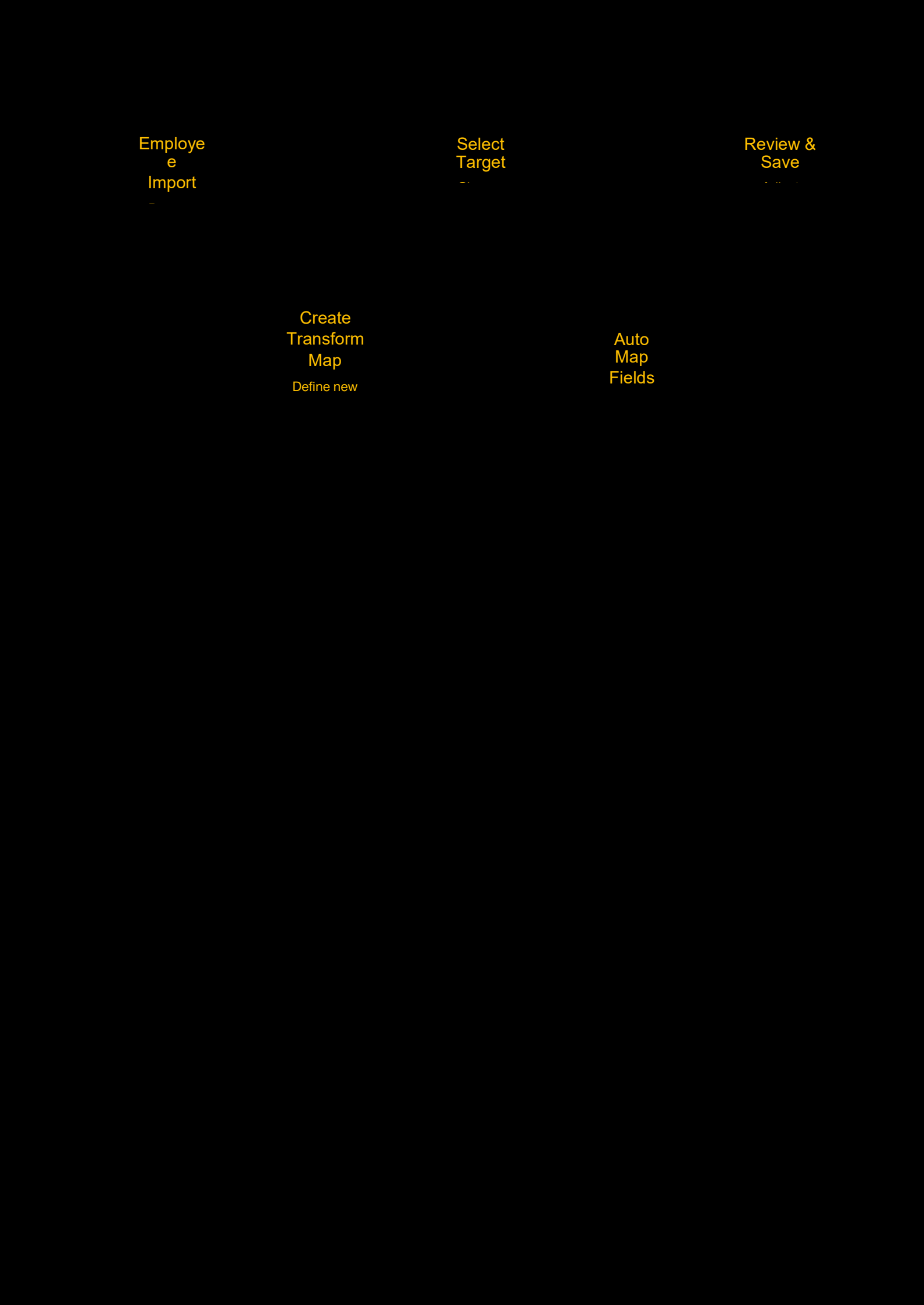
- Import Set label: Employee Import; Table: u_employee_import.
- Source file: Sample Spreadsheet.xlsx, Sheet number: 1, Header row: 1.
- Load spreadsheet data into the staging table and perform an initial review prior to running transforms.
- Reuse the Employee Import table for iterative test imports; clear or archive staging rows as needed.
- Plan the Import Set as the **staging area** for spreadsheet employee data.
- Confirm that the Import Set table is created with the label **Employee Import**.
- Verify that the table name is u_employee_import.
- Plan the spreadsheet upload using **Sample Spreadsheet.xlsx**.
- Select **Sheet 1** during the file upload process.
- Use **row 1 as the header row** so the column names are identified correctly.
- Verify that the uploaded data is available in the **Employee Import** staging table.
- Review the imported rows before starting the Transform operation.
- Check the Import Set status and confirm that the file is processed successfully before transforming the data.
- Plan to use the same Import Set table for subsequent test uploads and transformation activities.



Transform Map Planning

Design the Transform Map to map source fields to target fields and prepare mappings for execution.

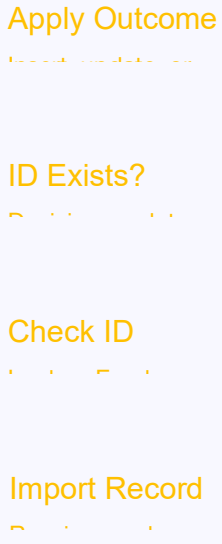
- Transform Map Name: Sample Spreadsheet Import.
- Source Table: u_employee_import; Target Table: u_employee_test.
- Plan source-to-target mappings for all five fields; enable Auto Map where column names match.
- Use Mapping Assist for remaining fields and review each mapping for type compatibility.
- Plan the Transform operation and document the expected Insert/Update behavior.
- Plan the Transform Map after creating the **Employee Import** and **Employee Test** tables.
- Confirm that the source table contains the required employee fields before transformation.
- Verify that the target table contains matching fields for the imported employee data.
- Plan the mapping for **Employee ID** as the key field for identifying employee records.
- Plan mappings for **Employee Name, Email, Department, and Location**.
- Review the field names and ensure the source and target fields are correctly connected.
- Use **Auto Map Matching Fields** when source and target column names are the same.
- Use **Mapping Assist** to manually connect fields that are not automatically matched.
- Review all field mappings before running the Transform operation.
- Record the expected transformation result: **new Employee IDs are inserted, while existing Employee IDs are updated when Coalesce is enabled**.


Employe
e
Import
Select
Target
Review &
Save
Create
Transform
Map
Define new
Auto
Map
Fields

Coalesce and Duplicate Management Planning

Establish duplicate prevention logic and decisions at transformation time.

- Assume repeated imports of similar data; use Coalesce to prevent duplicates.
- Set Employee ID as the Coalesce field to identify existing records.
- Behavior: existing Employee ID → Update existing record; new Employee ID → Insert new record.
- Plan to review transformation history and sample records to verify Coalesce results.



Testing and Validation Planning

Define test scenarios, expected outcomes, and the validation checklist to confirm correctness.

- Test the initial import and verify records in Employee Test.
- Scenario documented: upload 4 rows — 2 new Employee IDs, 2 existing IDs with changed data.
- Expected first run: 2 Inserted, 2 Updated. Repeating same spreadsheet: 0 Inserted, 0 Updated, 4 Ignored.
- Validate inserted and updated results, review transformation history, and confirm no duplicates exist in Employee Test.
- Verify that the uploaded spreadsheet contains the required five fields before starting the import.
- Confirm that the **Employee Import** table receives the uploaded spreadsheet data correctly.
- Check that the Transform Map uses the correct **source table and target table**.
- Verify that each source field is mapped to the corresponding target field.
- Test whether new Employee IDs create new records in the **Employee Test** table.
- Test whether existing Employee IDs update the existing employee records instead of creating duplicates.
- Verify that changed **Employee Name and Email** values are updated correctly for existing IDs.
- Review the **Transform History** to confirm the number of inserted, updated, and ignored records.
- Re-import the same spreadsheet and verify that previously processed records are ignored.
- Perform a final validation of the Employee Test table to confirm the imported data and duplicate- prevention behavior are correct.



Reporting and Dashboard Planning

Plan the reports and dashboard that will provide insights from the Employee Test table.

- Report 1 — Employees by Department: Source = Employee Test, Type = Pie Chart, Group by Department, Aggregation = Count.
- Report 2 — Employees by Location: Source = Employee Test, Type = Bar Chart, Group by Location, Aggregation = Count.
- Report 3 — Employee List Report: Source = Employee Test, Type = List with columns Employee ID, Name, Email, Department, Location.
- Dashboard: Employee Analytics Dashboards — include the three reports and configure sharing by Users, Groups, and Roles.
- Plan all reports using the **Employee Test** table as the report source.
- Ensure the reports use the imported employee records for analysis.
- Plan the Department report to show the distribution of employees across departments.
- Plan the Location report to show the distribution of employees across different locations.
- Use **Count** aggregation to determine the number of employee records in each group.
- Configure appropriate report formats so the grouped employee data is easy to understand.
- Include important employee fields in the Employee List Report for detailed record viewing.
- Verify that the three reports display the expected Employee Test data after import and transformation.
- Add the completed reports to the **Employee Analytics Dashboards** dashboard.
- Plan dashboard access by configuring appropriate **Users, Groups, and Roles** for sharing.

Validated Data Employee Test table validated

By Departme nt

Generate

Review & Save

Create Reports

By Location

Generate

Activity Sequence and Planning Summary

Present the ordered activities required to complete implementation and a concise academic summary of the planning phase.

- Sequence highlights: prepare spreadsheet → create staging and target tables → create and save Transform Map → configure Coalesce → execute transforms → validate and test → create reports → assemble dashboard → configure access → final review.
- Planning establishes the structure and sequence needed to implement a repeatable, auditable employee data import process using ServiceNow Import Sets and Transform Maps.



Implement Client Script & UI Policy (Incident)

Combined Project Document

Phase-3 Project Design + Phase-5 Project Development

Team ID: SWTID-2026-8014

Implement Client Script and UI Policy in ServiceNow – Incident Management

Phase-3 -- Project Design Phase

Project Design Document

Date	29.09.2026
Team Id	SWTID-2026--8014
Project Name	Implement client script and UI policy (Incident)
Project Guide	Mrs.S.Raja Shyamala M.Sc.,M.Phil.,B.Ed.,TET

Jeno Steffi A-Team leader	24721ER005
Jensi V	24721ER006
Kanika S	24721ER007
Renuga S	24721ER009

1. Project Design Overview

The project is designed to implement Client Script and UI Policy features on the ServiceNow Incident form. The objective is to improve form behaviour, enforce field requirements, control field visibility and read-only states, and provide client-side validation before an incident record is submitted.

- Design of the Incident form and relevant fields.
- Design of Client Scripts for client-side validation and dynamic form behaviour.
- Design of UI Policies for mandatory, visible, and read-only field control.
- Design of conditions based on Incident field values such as State, Priority, Category, and Assignment Group.
- Design of user-friendly form behaviour without unnecessary server-side processing.
- Design of test cases for new, existing, and updated Incident records.
- Design of validation and implementation verification.
- Design of a reusable approach for future Incident form enhancements.

2. System Architecture Design

The architecture separates the Incident form, client-side logic, UI Policy rules, and server-side Incident data. When a user opens or changes an Incident form, UI Policies and Client Scripts evaluate the current form values and apply the configured behaviour.

Source Layer: User interacts with the Incident form.

Client Layer: Client Scripts execute JavaScript on the client side for validation and dynamic behaviour.

UI Policy Layer: UI Policies evaluate conditions and control field visibility, mandatory status, and read-only status.

Data Layer: The Incident table (incident) stores the final incident record.

Validation Layer: Test scenarios verify that the configured rules behave correctly.

3. Data Flow Design

1. User opens the Incident form.

2. The form loads the configured UI Policies and Client Scripts.

3. UI Policy conditions are evaluated against Incident field values.

4. Matching UI Policies change field states such as mandatory, visible, or read-only.

5. Client Scripts respond to form load, field changes, or form submission as configured.

6. Client-side validation checks the required conditions before submission.

7. If validation succeeds, the Incident record is submitted.

8. The Incident record is stored in the Incident table.

9. Test results are reviewed against the expected form behaviour.

4. Incident Form Design

The Incident form is the primary user interface for the project. The design focuses on common Incident fields and their relationship to form behaviour.

Key fields considered: Number, Caller, Short Description, Description, Category, Subcategory, Impact, Urgency, Priority, Assignment Group, Assigned To, State, and Resolution information.

Field behaviour is controlled according to business conditions. For example, resolution-related fields can be required when an Incident reaches a resolution state, while assignment-related fields can be controlled based on the selected assignment group.

5. Client Script Design

Client Scripts are designed to execute JavaScript in the browser when specified Incident form events occur.

- onLoad Client Script: Initializes or adjusts form behaviour when the Incident form opens.
- onChange Client Script: Responds when a selected field value changes and updates related fields or messages.
- onSubmit Client Script: Performs client-side validation before the Incident form is submitted.

Example validation design: when an Incident is moved to a resolved state, the script checks whether the required resolution information has been entered. If the required information is missing, submission is prevented and the user receives a clear message.

Client Script design principles include minimal processing, clear user messages, field-level validation, and avoiding unnecessary server calls.

6. UI Policy Design

UI Policies are designed to control Incident form fields without requiring custom JavaScript for every field-state requirement.

- Mandatory Action: Makes a field mandatory when the policy condition is true.
- Visible Action: Shows or hides a field according to the condition.
- Read-only Action: Prevents editing when the condition is true.

Example UI Policy: When State indicates that the Incident is resolved, resolution-related fields become mandatory and selected operational fields can become read-only.

Another policy can control additional fields based on Category or other Incident values. UI Policy Actions should be configured for each affected field and tested in both matching and non-matching conditions.

7. Client Script and UI Policy Interaction

Client Scripts and UI Policies work together on the Incident form. UI Policies are preferred for straightforward field-state changes, while Client Scripts are used when conditional logic or validation requires scripting.

Design rule: avoid duplicating the same field-state logic in both mechanisms unless there is a clear requirement. This reduces conflicts and simplifies maintenance.

Execution should be tested when the form loads, when relevant fields change, and when the form is submitted.

8. Validation and Testing Design

Testing verifies both functional behaviour and user experience.

- Verify the Incident form loads correctly.
- Verify UI Policy conditions activate when expected.
- Verify mandatory fields become mandatory only under the configured condition.
- Verify fields become read-only when the policy condition is true.
- Verify fields are visible or hidden as designed.
- Verify Client Script messages appear for invalid input.
- Verify invalid Incident submissions are prevented when required conditions are not satisfied.
- Verify valid Incident records can be submitted successfully.
- Test both positive and negative conditions.
- Verify that changing a controlling field immediately produces the expected form behaviour.

9. Test Case Design

Test Case 1 – Form Load: Open an Incident and verify initial field states.

Test Case 2 – UI Policy Activation: Set the configured controlling field value and verify mandatory/read-only/visible actions.

Test Case 3 – UI Policy Deactivation: Change the controlling value back and verify the fields return to the configured normal state.

Test Case 4 – Client onChange: Change the relevant Incident field and verify the dependent behaviour.

Test Case 5 – Client onSubmit: Attempt to submit an Incident with missing required information and verify that submission is blocked.

Test Case 6 – Valid Submission: Provide valid information and verify that the Incident can be submitted.

Test Case 7 – Regression: Verify unrelated Incident fields and standard form behaviour continue to work.

10. Implementation and Validation Summary

The implementation uses the Incident table as the target record structure and combines UI Policies with Client Scripts to control client-side form behaviour. UI Policies handle field-state requirements such as mandatory, visible, and read-only states, while Client Scripts handle dynamic validation and logic.

Validation is performed through form-load, field-change, and form-submit scenarios. The final design should provide predictable Incident form behaviour, clear user feedback, and maintainable configuration.

11. Design Components Summary

Incident Form — Main user interface for incident data entry.

UI Policy — Controls mandatory, visible, and read-only field states.

UI Policy Action — Defines the field-level action applied by a UI Policy.

Client Script — Implements client-side logic and validation.

Incident Table — Stores Incident records.

Testing — Confirms expected behaviour under different field conditions.

Documentation — Records configuration, conditions, scripts, and expected results.

Conclusion:

This project provides a structured design for implementing Client Scripts and UI Policies on the ServiceNow Incident form. The design separates field-state control from scripted validation, uses condition-based behaviour, and includes systematic testing for form load, field changes, and submission. The approach is intended to improve data quality, consistency, and usability while keeping the configuration understandable and reusable.

Implement Client Script & UI Policy (Incident)

Phase-5 – Project Development Phase

Date	28.09.2026
Team Id	-2026-8014
Project Name	**Implement Client Script & UI Policy (Incident)**
Project Guide Name	Dr.K. Karpagam,Mrs. S. Raja Syamala, M.Sc., M.Phil.,B. Ed., TET

Team Members:

Jeno Steffi A -Team Leader	24721ER005
Jensi V	24721ER006
KanikaS	24721ER007
Renuga S	24721ER009

This phase converts planned concepts into concrete ServiceNow configurationand testing steps. Activities focus on developmentartifacts on the Incidentform and their verification rather than planning decisions.

Set up a controlled ServiceNow development environment and verify prerequisites before configuration.

1. Obtain a ServiceNow development/learning instance (Personal Developer Instance).
2. Log in with a user account that has the admin role (or equivalent) to create Client Scripts and UI Policies.
3. Open the Incident table: All > Incident > Create New / Open, and confirm the form loads correctly.
4. Open System Definition > Client Scripts to confirm the module is available.
5. Open System UI > UI Policies to confirm the module is available.
6. Confirm that the required Incident fields exist: Number, Caller, Category, Short description, State, Priority.
7. Use the Incident form for testing; avoid changing existing out-of-the-box scripts and policies.
8. Give every new script and policy a clear name so it can be identified and tested easily.
9. Verify that the instance is accessible and ready before beginning development.

3. Incident Form Requirements

Define the behavior needed on the Incident form. Keep each requirement small and measurable so it can be mapped directly to either a Client Script or a UI Policy.

No.	Requirement	Implemented Using	Trigger
1	Show a guidance message when the Incident form opens	Client Script (onLoad)	Form load
2	Alert the user when Priority is changed to 1 - Critical	Client Script (onChange)	Priority field
3	Prevent saving a Critical incident without a Short description	Client Script (onSubmit)	Submit / Save
4	Make Resolution code and Resolution notes mandatory and visible	UI Policy	State = Resolved
5	Make On hold reason mandatory	UI Policy	State = On Hold

Fields Used

- **Short description** - text summary of the incident.
- **State** - New, In Progress, On Hold, Resolved, Closed.
- **Priority** - 1 - Critical to 5 - Planning.
- **Resolution code / Resolution notes** - closure information entered when the incident

is resolved.

- **On hold reason** - reason shown when the incident is placed On Hold.

4. Client Script Development

Develop client-side scripts that run in the browser to control the Incident form. Navigate to System Definition > Client Scripts > New.

Client Script 1 - Form Guidance (onLoad)

- **Name**: Incident Form Guidance
- **Table**: Incident [incident] **Type**: onLoad; **Active**: checked

```
function onLoad() {
  g_form.addInfoMessage('Please fill in all mandatory fields before submitting.');
}
```

Client Script 2 - Priority Alert (onChange)

- **Name**: Critical Priority Alert
- **Table**: Incident [incident] **Type**: onChange; **Field name**: Priority

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }
  if (newValue == '1') {
    alert('Priority 1 - Critical selected. Provide complete details.');
  }
}
```

Client Script 3 - Submit Validation (onSubmit)

- **Name**: Validate Critical Incident
- **Table**: Incident [incident] **Type**: onSubmit

```
function onSubmit() {
  var priority = g_form.getValue('priority');
  var desc = g_form.getValue('short_description');
  if (priority == '1' && desc.length < 10) {
    g_form.addErrorMessage('Enter a Short description (min 10 characters) for a Critical incident.');
    return false;
  }
  return true;
}
```

- Save each Client Script and confirm it appears in the Client Scripts list for the Incident table.

5. UI Policy Development

Develop UI Policies to change field behavior (mandatory, visible, read-only) without writing

code. Navigate to System UI > UI Policies > New.

UI Policy 1 - Resolution Fields Mandatory

- **Short description**: Resolution fields mandatory when Resolved
- **Table**: Incident [incident], **Active**: checked; **On load**: checked; **Reverse if false**: checked
- **Condition**: State is Resolved
- **UI Policy Actions**

  Resolution code - Mandatory: True, Visible: True

  Resolution notes - Mandatory: True, Visible: True

UI Policy 2 - On Hold Reason Mandatory

- **Short description**: On hold reason mandatory when On Hold
- **Table**: Incident [incident], **Active**: checked; **On load**: checked; **Reverse if false**: checked
- **Condition**: State is On Hold
- **UI Policy Action**: On hold reason - Mandatory: True, Visible: True

Client Script vs UI Policy

Point	Client Script	UI Policy
Approach	Written in JavaScript	Configured with conditions and actions (no-code)
Best for	Alerts, validation, custom logic	Mandatory, visible and read-only field control
Reverse behavior	Must be coded manually	Reverse if false option
Runs on	Load, change, submit, cell edit	Form load and field change

6. Testing & Validation

Run each scenario on the Incident form and compare the result with the expected outcome. Record any difference and correct the script or policy before final verification.

Scenario	Action	Expected Outcome
Open Incident form	Open a new or existing Incident	Info message is shown at the top of the form
Priority changed to 1	Set Priority to 1 - Critical	Alert about Critical priority is displayed
Priority changed to 3	Set Priority to 3 - Moderate	No alert is displayed
Submit Critical incident, short text	Priority 1 with Short description under 10 characters, then Save	Error message shown; record is not saved

Submit Critical incident, valid text	Priority 1 with valid Short description, then Save	Record is saved successfully
State = Resolved	Set State to Resolved	Resolution code and notes become visible and mandatory
State changed back	Change State to In Progress	Resolution fields return to normal (Reverse if false)
State = On Hold	Set State to On Hold	On hold reason becomes mandatory

7. Development Verification and Result

This section confirms that all development activities were completed and behave as required on the Incident form.

1. Three Client Scripts (onLoad, onChange, onSubmit) are created, active and saved.
2. Two UI Policies are created with conditions, actions and Reverse if false enabled.
3. All test scenarios were executed and produced the expected outcomes.
4. Mandatory fields are enforced correctly for Resolved and On Hold states.
5. Critical incidents cannot be saved without a proper Short description.
6. Existing Incident records and out-of-the-box behavior are not affected.

Result

The Incident form now responds dynamically to user actions. Client Scripts provide guidance, alerts and validation, while UI Policies control mandatory and visible fields based on the State value. The project is ready for the next phase.

Implement client script and UI policy (Incident)

Phase-8 -- Project Demonstration Phase

Date	29.09.2026
Team Id	SWTID-2026-8014
Project Name	Implement client script and UI policy (Incident)
Project Guide Name	Mrs. S. Raja Shyamala,M.Sc., M.Phil., B.Ed.,TET

Team Members:

Jeno Steffi A - Team Leader	24721ER005
Jensi V	24721ER006
Kanika S	24721ER007
Renuga S	24721ER009

1. Project Demonstration Overview

This phase demonstrates operation, configuration, and verification tasks required to validate the employee import implementation. The demonstration covers preparation of the source spreadsheet, loading to the Import Set (staging) table, review and configuration of the Transform Map, execution of transforms, Coalesce configuration, verification of inserted and updated records, repeat-import behavior, Transform History review, creation of reports, and dashboard assembly.

- Demonstration of spreadsheet preparation
- Demonstration of data loading into Employee Import
- Demonstration of staging data verification
- Demonstration of Transform Map configuration and mapping
- Demonstration of transformation execution and Coalesce
- Demonstration of new record insertion and existing-record update
- Demonstration of repeated import behavior and Transform History
- Demonstration of reports and Employee Analytics Dashboards
- Final project validation

Complete demonstration flow: prepare spreadsheet → load spreadsheet → verify staging rows in Employee Import → open Sample Spreadsheet Import transform map → verify field maps and Coalesce → run Transform → review Employee Test table → validate Transform History → create and run reports → assemble dashboard → final validation.

2. Demonstration Objectives

The demonstration objectives enumerate discrete verification goals for the project. Each objective aligns to documented features and outcomes from the completed project.

1. Demonstrate the complete spreadsheet import process.
2. Demonstrate the Employee Import staging table.
3. Demonstrate the Employee Test target table.
4. Demonstrate the Transform Map configuration.
5. Demonstrate source-to-target field mapping.
6. Demonstrate successful transformation.
7. Demonstrate Coalesce behavior.
8. Demonstrate insertion of new employee records.
9. Demonstrate updating existing employee records.
10. Demonstrate repeated spreadsheet import.
11. Demonstrate Transform History results.
12. Demonstrate report creation and execution.
13. Demonstrate dashboard creation.
14. Demonstrate adding reports to the dashboard.
15. Verify the final project output.

Each objective is verified by the demonstration procedures and the documented results in the project file.

3. Demonstration Environment

This section lists the ServiceNow artifacts used in the demonstration and their roles. No external technologies or systems are involved beyond the documented ServiceNow components.

Component	Role / Description
Spreadsheet file	Source Excel file (Sample Spreadsheet.xlsx) containing employee records used for import.
Employee Import (u_employee_import)	Import Set (staging) table that receives uploaded spreadsheet rows prior to transformation.
Employee Test (u_employee_test)	Target table storing transformed and persisted employee records.
Transform Map	Sample Spreadsheet Import transform map performing source→target mapping and transformation logic.
Field Maps	Individual source-to-target field mapping definitions including Coalesce settings.
Coalesce	Field-level identification used to match existing Employee Test records to avoid duplicates.
Transform History	System record of transform runs showing uploaded/inserted/updated/ignored counts and status.
Reports	Saved reports built on Employee Test: Employees by Department, Employees by Location, Employee List Report.
Employee Analytics Dashboards	Dashboard containing the three reports for consolidated viewing.

4. Demonstration of Spreadsheet

Preparation

Procedure to prepare the source spreadsheet for import. This is a documentation-level demonstration of the preparation steps used in the project.

1. Open a new Google Spreadsheet (or equivalent authoring tool).
2. Create sample employee data rows consistent with the project column structure.
3. Ensure required columns are present and named exactly as expected by mapping: Employee ID, Employee Name, Email, Department, Location.
4. Validate field values for format (e.g., email address format) and remove extraneous formatting.
5. Download the file as Excel: Sample Spreadsheet.xlsx and retain the original for traceability.

Spreadsheet field documentation:

Field	Purpose
Employee ID	Identifies the employee
Employee Name	Stores employee name
Email	Stores employee email
Department	Stores department
Location	Stores employee location

5. Demonstration of Employee Test

Table (Target)

The Employee Test table is the transform target where validated employee records are persisted. The table and field definitions are based on the project document.

- The target table exists.
- The required fields are configured.
- Employee records can be stored in the table.
- Transformed records appear after a successful transformation.
- New employee records can be inserted.
- Existing employee records can be updated.
- The table provides the source data for the project's reports.
- The table forms the main data layer between transformation and reporting.

Component	Configuration
Table Label	Employee Test
Table Name	u_employee_test
Table Type	Custom target table
Purpose	Stores transformed employee records
Source	Employee Import staging table
Transformation	Sample Spreadsheet Import Transform Map
Main Identifier	Employee ID

Required Fields	Employee ID, Employee Name, Email, Department, Location

Required target fields (documented): Employee ID, Employee Name, Email, Department, Location. These fields receive transformed values according to the field mapping defined in the Transform Map.

6. Demonstration of Employee Import

Table (Staging)

The **Employee Import** table is used as a temporary staging table. It stores the employee data from the spreadsheet before sending it to the Transform Map.

Demonstration Steps

1. Open **System Import Sets → Load Data**.
2. Upload **Sample Spreadsheet.xlsx**.
3. Select **Sheet 1**.
4. Select **Row 1** as the header row.
5. Select the **Employee Import** table.
6. Verify the table name: u_employee_import.
7. Load the spreadsheet data.
8. Open the Employee Import table.
9. Check the imported employee records.
10. Verify the **Employee ID**.
11. Verify the **Employee Name**.
12. Verify the **Email**.
13. Verify the **Department**.
14. Verify the **Location**.
15. Confirm that the records are ready for transformation.

Employee Import Fields

Field	Purpose
Employee ID	Identifies the employee
Employee Name	Stores employee name
Email	Stores email

Department	Stores department
Location	Stores location

7. Demonstration of Transform Map

Transform Map configuration is the core mapping artifact. The project used a specific Transform Map named "Sample Spreadsheet Import" to map staging fields to Employee Test target fields.

1. Open System Import Sets → Transform Maps and select "Sample Spreadsheet Import".
2. Verify Transform Map name: Sample Spreadsheet Import.
3. Verify Source Table: Employee Import (u_employee_import).
4. Verify Target Table: Employee Test (u_employee_test).
5. Use Auto Map Matching Fields and then open Mapping Assist to refine mappings.
6. Save the Transform Map after confirming mappings and Coalesce settings.

Configuration	Value
Transform Map	Sample Spreadsheet Import
Source Table	Employee Import (u_employee_import)
Target Table	Employee Test (u_employee_test)
Mapping	Auto Map / Mapping Assist

8. Demonstration of Field Mapping

Field mapping connects the fields in the **Employee Import** table with the fields in the **Employee Test** table.

Field Mapping

Source Field	Target Field
Employee ID	Employee ID
Employee Name	Employee Name
Email	Email
Department	Department
Location	Location

Demonstration Steps

1. Open the **Sample Spreadsheet Import** Transform Map.
2. Check the source table: **Employee Import**.
3. Check the target table: **Employee Test**.
4. Open the field mapping section.
5. Verify the Employee ID mapping.
6. Verify the Employee Name mapping.
7. Verify the Email mapping.
8. Verify the Department mapping.
9. Verify the Location mapping.
10. Run the transformation.
11. Check the Employee Test table to confirm the mapped data.

9. Demonstration of Transform

Execution and Results

Procedure to execute the transform and verify results. The project document records the transform ran successfully and produced the documented counts.

1. Open Transform Map: Sample Spreadsheet Import.
2. Select Transform action.
3. Choose the appropriate Import Set instance (the recently loaded Employee Import records).
4. Ensure Sample Spreadsheet Import Transform Map is selected.
5. Click Transform and monitor the Transform progress until completion.
6. Open Employee Test (u_employee_test) and validate transformed records appear with expected field values.

Documented outcome (project file): transformation completed successfully; Employee Test contains the imported records with both inserted and updated rows as detailed in the project documentation.

Verification Item	Result / Comment
Transform completion status	Successful (as documented)
Employee Test records created/updated	See Target Table Validation and Transform History documented counts

ImplementClient Script & UI Policy (Incident)

Phase-2 -- Requirement Analysis Phase

Date	29.09.2026
Team Id	SWTID-20268014
Project Name	ImplementClient Script & UI Policy (Incident)
Project Guide Name	Mrs.S.RajaSyamala, M.Sc., M.Phil.,B.Ed., TET

Team Members:

Jeno Steffi A-Team Leader	24721ER005
Jensi V	24721ER006
KanikaS	24721ER007
RenugaS	24721ER009

1. Requirement Analysis Overview

Primary objective: customise the ServiceNow **Incident** form using **Client Scripts** and **UI Policies** so that users enter complete, valid data and the form behaves dynamically (mandatory, visible and read-only fields) based on the incident State and Priority.

- **Target table:** Incident [incident] – the form used by the service desk to log, work and resolve incidents.
- **Client Scripts:** JavaScript that runs in the user's browser on form events – onLoad, onChange, onSubmit and onCellEdit.
- **UI Policies:** condition-based, mostly no-code rules that make fields Mandatory, Visible or Read-only (with optional scripts).
- **Validation:** block incomplete or invalid submissions and give the user clear on-screen guidance.
- **Testing:** every Client Script and UI Policy is verified with defined test cases (Section 8).
- **Governance:** clear naming standard, correct roles for administration, and awareness that client-side logic is not a security control.

2. Functional Requirements

Client Script requirements

- Show a guidance message when a **new** incident form is opened (onLoad).
- Warn the user immediately when the incident Priority becomes **1 – Critical** (onChange on Priority).
- Alert the agent when the selected Caller is a **VIP** user (onChange on Caller).
- Stop the form from being saved when the Short description is missing or shorter than 10 characters (onSubmit).
- Prevent an incident from being set to **Closed** through inline list editing (onCellEdit on State).
- Client Scripts must be created on the Incident table, be Active and use UI Type = All.

UI Policy requirements

- Make **On hold reason** visible and mandatory when State is **On Hold**.
- Make **Resolution code** and **Resolution notes** mandatory when State is **Resolved**.
- Make **Assigned to** mandatory when State is **In Progress**.
- Make **Description** mandatory when Priority is **1 – Critical**.
- Make key fields **read-only** when State is **Closed**.
- Use **Reverse if false** so fields return to their normal behaviour when the condition no longer applies.

Common requirements

- Changes must work on the standard Incident form for users with the itil role.
- Client Script and UI Policy must not conflict; where both act on one field, the UI Policy result takes precedence because it runs after client scripts.
- Each script and policy must have a meaningful name, short description and Active flag.
- All behaviour must be tested by creating, editing, saving and closing test incidents.

3. Data Requirements

The customisation uses only standard Incident fields, so no new table or field is required. The fields used are listed below.

Field label	Field name	Type	Used by
Caller	caller_id	Reference (User)	onChange – VIP warning
Short description	short_description	String	onSubmit – length validation
Description	description	String	UI Policy – mandatory for P1
Impact / Urgency	impact / urgency	Choice	Calculate Priority
Priority	priority	Choice (calculated)	onChange – P1 warning; UI Policy condition
State	state	Choice	UI Policy conditions; onCellEdit
On hold reason	hold_reason	Choice	UI Policy – On Hold
Assigned to	assigned_to	Reference (User)	UI Policy – In Progress
Resolution code	close_code	Choice	UI Policy – Resolved
Resolution notes	close_notes	String	UI Policy – Resolved

State values: New = 1, In Progress = 2, On Hold = 3, Resolved = 6, Closed = 7, Canceled = 8.
Priority values: 1 – Critical, 2 – High, 3 – Moderate, 4 – Low, 5 – Planning.

- Field names (back-end names) must be used in scripts, not field labels.
- Choice values are stored as strings, so comparisons such as newValue == '1' must use quotes or loose equality.
- Test records should use clearly marked short descriptions (for example, prefix "TEST") so they can be found and cleaned up.

4. Client Script Requirements

Path: **All > System Definition > Client Scripts > New**. Common settings for every script: Table = **Incident [incident]**, Active = true, UI Type = **All**.

Name	Type	Field	Purpose
Incident – New Record Guidance	onLoad	–	Show guidance message on a new incident
Incident – Priority 1 Warning	onChange	Priority	Warn when priority becomes Critical
Incident – VIP Caller Alert	onChange	Caller	Alert agent if caller is a VIP
Incident – Validate Short Description	onSubmit	–	Block save if short description is too short
Incident – Block Close from List	onCellEdit	State	Stop closing from list inline edit

4.1 onLoad – Incident – New Record Guidance

```
function onLoad() {
    // Runs every time the form loads
    if (g_form.isNewRecord()) {
        g_form.addInfoMessage(
            'Please enter complete details so the support team can resolve it faster.');
```

```
    }
}
```

4.2 onChange – Incident – Priority 1 Warning (Field: Priority)

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    if (newValue == '1') {
        g_form.showFieldMsg('priority',
            'Critical incident - inform the Incident Manager immediately.', 'error');
    } else {
        g_form.hideFieldMsg('priority', true);
    }
}
```

4.3 onChange – Incident – VIP Caller Alert (Field: Caller)

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    g_form.getReference('caller_id', function (caller) {
        if (caller.vip == 'true') {
            g_form.showFieldMsg('caller_id',
                'VIP caller - handle with high priority.', 'info');
        } else {
            g_form.hideFieldMsg('caller_id', true);
        }
    });
}
```

4.4 onSubmit – Incident – Validate Short Description

```
function onSubmit() {
    var shortDesc = g_form.getValue('short_description').trim();
    if (shortDesc.length < 10) {
        g_form.showFieldMsg('short_description',
            'Short description must be at least 10 characters.', 'error');
        return false;  // stops the save
    }
    return true;
}
```

4.5 onCellEdit – Incident – Block Close from List (Field: State)

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAndClose = true;
    if (newValue == '7') {   // 7 = Closed
        alert('Incidents cannot be closed from the list. Open the record.');
        saveAndClose = false;
    }
    callback(saveAndClose);
}
```

5. UI Policy Requirements

Path: **All > System UI > UI Policies > New**. Common settings: Table = **Incident [incident]**, Active = true, **On load** = true, **Reverse if false** = true.

UI Policy name	Condition	UI Policy Actions

Incident – On Hold Reason Required	State is On Hold	On hold reason: Mandatory = True, Visible = True
Incident – Resolution Details Required	State is Resolved	Resolution code: Mandatory = True; Resolution notes: Mandatory = True
Incident – Assigned To Required	State is In Progress	Assigned to: Mandatory = True
Incident – P1 Description Required	Priority is 1 - Critical	Description: Mandatory = True
Incident – Closed Read Only	State is Closed	Short description, Description, Caller, Category, Impact, Urgency, Assignment group: Read only = True

5.1 UI Policy script (optional) – Incident – P1 Description Required

Tick **Run scripts** on the policy, then add the following in the Script tab.

Execute if true:

```
function onCondition() {
   g_form.addInfoMessage('Priority 1 incident: Description is mandatory.');
}
```

Execute if false:

```
function onCondition() {
   g_form.clearMessages();
}
```

5.2 Key UI Policy behaviours

- **Reverse if false** undoes the actions automatically when the condition stops being true.
- **On load** applies the policy when an existing record is opened, not only when a value changes.
- UI Policy Actions control only **Mandatory, Visible and Read only** anything more needs a script.
- UI Policies run in the browser only; they do not apply to server-side imports or list editing.

6. Implementation Steps

6.1 Create a Client Script

- **Step 1:** Log in to the ServiceNow instance as **admin**.
- **Step 2:** In the filter navigator search for **Client Scripts** and open **System Definition > Client Scripts**.
- **Step 3:** Click **New** and enter the Name, Table = Incident, UI Type = All and choose the Type (onLoad / onChange / onSubmit / onCellEdit).
- **Step 4:** For onChange and onCellEdit select the **Field name** (for example Priority, Caller or State).
- **Step 5:** Make sure **Active** is ticked, paste the script from Section 4 into the Script box and click **Submit**.
- **Step 6:** Open **Incident > Create New** and perform the action that triggers the script to confirm it works.

6.2 Create a UI Policy

- **Step 1:** Open **System UI > UI Policies** and click **New**.

- **Step 2:** Set Table = Incident, enter a Short description, tick **Active**, **On load** and **Reverse if false**.
- **Step 3:** In **When to Apply** build the condition, for example State → is → On Hold.
- **Step 4:** Click **Submit** (or Save), then open the policy again and use the **UI Policy Actions** related list to click **New**.
- **Step 5:** Choose the **Field name** and set Mandatory / Visible / Read only to True, False or Leave alone, then **Submit**.
- **Step 6:** Repeat for the remaining policies in Section 5 and test each on the Incident form.

6.3 Implementation checklist

- 5 Client Scripts created and Active (1 onLoad, 2 onChange, 1 onSubmit, 1 onCellEdit).
- 5 UI Policies created with conditions, actions and Reverse if false.
- Test incidents created and every test case in Section 8 executed.
- Screenshots captured for the project report (script list, policy list, and form behaviour).

7. Client Script vs UI Policy and Best Practices

Point	Client Script	UI Policy
Coding	JavaScript is required	No code for standard actions; script optional
Trigger	onLoad, onChange, onSubmit, onCellEdit	Condition true / false (on load and on change)
Best used for	Validation, messages, blocking submit, reference look-ups	Mandatory, Visible and Read-only behaviour
Stop form save	Yes – onSubmit returns false	No
List editing	Yes – onCellEdit	No
Execution order	Runs first	Runs after client scripts, so it wins on conflicts

Best practices

- Prefer a **UI Policy** when the requirement is only mandatory, visible or read-only; use a Client Script when logic is needed.
- Always check **isLoading** in onChange scripts so they do not run while the form is loading.
- Use the **callback** version of getReference and avoid synchronous server calls that slow the form.
- Do not manipulate the DOM or use jQuery; use only supported **g_form** and **g_user** APIs.
- Client-side logic can be bypassed – protect data with **ACLs, Data Policies or Business Rules** as well.
- Use a consistent naming pattern (Table – Purpose) and always add a description.
- Test in a sub-production instance and move changes using **Update Set**.

8. Validation and Testing Requirements

ID	Scenario	Steps	Expected result
TC01	onLoad message	Incident > Create New	Blue guidance message appears at top of the form

TC02	Priority 1 warning	Set Impact = 1 - High and Urgency = 1 - High	Priority = 1 - Critical and red message shows under Priority
TC03	Priority warning clears	Change Impact or Urgency so priority is not 1	Message under Priority is removed
TC04	VIP caller	Select a caller whose VIP box is ticked	Info message “VIP caller” shows under Caller
TC05	Short description validation	Enter “Test” and click Submit	Save blocked; error message under Short description
TC06	On Hold policy	Set State = On Hold	On hold reason appears and is mandatory; Save is blocked until filled
TC07	Resolved policy	Set State = Resolved	Resolution code and Resolution notes become mandatory
TC08	In Progress policy	Set State = In Progress	Assigned to is mandatory
TC09	Reverse if false	Change State from On Hold back to In Progress	On hold reason is no longer mandatory
TC10	P1 description	Create a Priority 1 incident with empty Description	Description is mandatory; asterisk shown
TC11	Closed read-only	Open a Closed incident	Key fields are read-only
TC12	onCellEdit	In the Incident list edit State to Closed	Alert shown; change is not saved

- Each test case is marked **Pass / Fail** with a screenshot as evidence.
- If a script does not fire, check that it is **Active**, the correct **Table / Field name / UI Type** is selected, and view the browser console for JavaScript errors.
- Regression-test the standard Incident form after all changes to confirm nothing else is broken.

9. Learning Requirements

- Understand the difference between **client-side** and **server-side** scripting in ServiceNow.
- Learn the four Client Script types: onLoad, onChange, onSubmit and onCellEdit.
- Learn the parameters of onChange: control, oldValue, newValue, isLoading, isTemplate.
- Learn the **g_form** methods: getValue, setValue, isNewRecord, showFieldMsg, hideFieldMsg, addInfoMessage, clearMessages, getReference.
- Understand UI Policy conditions, actions, **Reverse if false** and **On load**
- Understand when to use a UI Policy and when to use a Client Script.
- Practise testing and debugging form behaviour using test incidents and the browser console.
- Understand why client-side logic must be backed by server-side controls.

10. Requirement Summary

Overall Requirement Flow

Incident Form → Client Script (onLoad / onChange / onSubmit) → UI Policy (Mandatory / Visible / Read-only) → Validation → Save Incident → Testing

Summary Points

- Customise the Incident form using Client Scripts and UI Policies.
- Use onLoad to guide users when a new incident is created.

- Use onChange to react to Priority and Caller changes.
- Use onSubmit to validate the Short description before saving.
- Use onCellEdit to stop incidents being closed from the list.
- Use UI Policies to control mandatory, visible and read-only fields by State and Priority.
- Use Reverse if false so form behaviour returns to normal automatically.
- Test every script and policy with defined test cases.
- Follow best practices: naming, isLoading check, no DOM code and server-side protection.
- The result is a cleaner, more accurate and easier-to-use Incident form.

Implement Client Script and UI Policy in ServiceNow – Incident Management

Phase-3 -- Project Design Phase

Project Design Document

Date	29.09.2026
Team Id	SWTID-2026--8014
Project Name	Implement client script and UI policy (Incident)
Project Guide	Mrs.S.Raja Shyamala M.Sc.,M.Phil.,B.Ed.,TET

Jeno Steffi A-Team leader	24721ER005
Jensi V	24721ER006
Kanika S	24721ER007
Renuga S	24721ER009

1. Project Design Overview

The project is designed to implement Client Script and UI Policy features on the ServiceNow Incident form. The objective is to improve form behaviour, enforce field requirements, control field visibility and read-only states, and provide client-side validation before an incident record is submitted.

- Design of the Incident form and relevant fields.
- Design of Client Scripts for client-side validation and dynamic form behaviour.
- Design of UI Policies for mandatory, visible, and read-only field control.
- Design of conditions based on Incident field values such as State, Priority, Category, and Assignment Group.
- Design of user-friendly form behaviour without unnecessary server-side processing.
- Design of test cases for new, existing, and updated Incident records.
- Design of validation and implementation verification.
- Design of a reusable approach for future Incident form enhancements.

2. System Architecture Design

The architecture separates the Incident form, client-side logic, UI Policy rules, and server-side Incident data. When a user opens or changes an Incident form, UI Policies and Client Scripts evaluate the current form values and apply the configured behaviour.

Source Layer: User interacts with the Incident form.

Client Layer: Client Scripts execute JavaScript on the client side for validation and dynamic behaviour.

UI Policy Layer: UI Policies evaluate conditions and control field visibility, mandatory status, and read-only status.

Data Layer: The Incident table (incident) stores the final incident record.

Validation Layer: Test scenarios verify that the configured rules behave correctly.

3. Data Flow Design

1. User opens the Incident form.

2. The form loads the configured UI Policies and Client Scripts.

3. UI Policy conditions are evaluated against Incident field values.

4. Matching UI Policies change field states such as mandatory, visible, or read-only.

5. Client Scripts respond to form load, field changes, or form submission as configured.

6. Client-side validation checks the required conditions before submission.

7. If validation succeeds, the Incident record is submitted.

8. The Incident record is stored in the Incident table.

9. Test results are reviewed against the expected form behaviour.

4. Incident Form Design

The Incident form is the primary user interface for the project. The design focuses on common Incident fields and their relationship to form behaviour.

Key fields considered: Number, Caller, Short Description, Description, Category, Subcategory, Impact, Urgency, Priority, Assignment Group, Assigned To, State, and Resolution information.

Field behaviour is controlled according to business conditions. For example, resolution-related fields can be required when an Incident reaches a resolution state, while assignment-related fields can be controlled based on the selected assignment group.

5. Client Script Design

Client Scripts are designed to execute JavaScript in the browser when specified Incident form events occur.

- onLoad Client Script: Initializes or adjusts form behaviour when the Incident form opens.
- onChange Client Script: Responds when a selected field value changes and updates related fields or messages.
- onSubmit Client Script: Performs client-side validation before the Incident form is submitted.

Example validation design: when an Incident is moved to a resolved state, the script checks whether the required resolution information has been entered. If the required information is missing, submission is prevented and the user receives a clear message.

Client Script design principles include minimal processing, clear user messages, field-level validation, and avoiding unnecessary server calls.

6. UI Policy Design

UI Policies are designed to control Incident form fields without requiring custom JavaScript for every field-state requirement.

- Mandatory Action: Makes a field mandatory when the policy condition is true.
- Visible Action: Shows or hides a field according to the condition.
- Read-only Action: Prevents editing when the condition is true.

Example UI Policy: When State indicates that the Incident is resolved, resolution-related fields become mandatory and selected operational fields can become read-only.

Another policy can control additional fields based on Category or other Incident values. UI Policy Actions should be configured for each affected field and tested in both matching and non-matching conditions.

7. Client Script and UI Policy Interaction

Client Scripts and UI Policies work together on the Incident form. UI Policies are preferred for straightforward field-state changes, while Client Scripts are used when conditional logic or validation requires scripting.

Design rule: avoid duplicating the same field-state logic in both mechanisms unless there is a clear requirement. This reduces conflicts and simplifies maintenance.

Execution should be tested when the form loads, when relevant fields change, and when the form is submitted.

8. Validation and Testing Design

Testing verifies both functional behaviour and user experience.

- Verify the Incident form loads correctly.
- Verify UI Policy conditions activate when expected.
- Verify mandatory fields become mandatory only under the configured condition.
- Verify fields become read-only when the policy condition is true.
- Verify fields are visible or hidden as designed.
- Verify Client Script messages appear for invalid input.
- Verify invalid Incident submissions are prevented when required conditions are not satisfied.
- Verify valid Incident records can be submitted successfully.
- Test both positive and negative conditions.
- Verify that changing a controlling field immediately produces the expected form behaviour.

9. Test Case Design

Test Case 1 – Form Load: Open an Incident and verify initial field states.

Test Case 2 – UI Policy Activation: Set the configured controlling field value and verify mandatory/read-only/visible actions.

Test Case 3 – UI Policy Deactivation: Change the controlling value back and verify the fields return to the configured normal state.

Test Case 4 – Client onChange: Change the relevant Incident field and verify the dependent behaviour.

Test Case 5 – Client onSubmit: Attempt to submit an Incident with missing required information and verify that submission is blocked.

Test Case 6 – Valid Submission: Provide valid information and verify that the Incident can be submitted.

Test Case 7 – Regression: Verify unrelated Incident fields and standard form behaviour continue to work.

10. Implementation and Validation Summary

The implementation uses the Incident table as the target record structure and combines UI Policies with Client Scripts to control client-side form behaviour. UI Policies handle field-state requirements such as mandatory, visible, and read-only states, while Client Scripts handle dynamic validation and logic.

Validation is performed through form-load, field-change, and form-submit scenarios. The final design should provide predictable Incident form behaviour, clear user feedback, and maintainable configuration.

11. Design Components Summary

Incident Form — Main user interface for incident data entry.

UI Policy — Controls mandatory, visible, and read-only field states.

UI Policy Action — Defines the field-level action applied by a UI Policy.

Client Script — Implements client-side logic and validation.

Incident Table — Stores Incident records.

Testing — Confirms expected behaviour under different field conditions.

Documentation — Records configuration, conditions, scripts, and expected results.

Conclusion:

This project provides a structured design for implementing Client Scripts and UI Policies on the ServiceNow Incident form. The design separates field-state control from scripted validation, uses condition-based behaviour, and includes systematic testing for form load, field changes, and submission. The approach is intended to improve data quality, consistency, and usability while keeping the configuration understandable and reusable.

Implement Client Script & UI Policy (Incident)

Phase-5 – Project Development Phase

Date	28.09.2026
Team Id	-2026-8014
Project Name	**Implement Client Script & UI Policy (Incident)**
Project Guide Name	Dr.K. Karpagam,Mrs. S. Raja Syamala, M.Sc., M.Phil.,B. Ed., TET

Team Members:

Jeno Steffi A -Team Leader	24721ER005
Jensi V	24721ER006
KanikaS	24721ER007
Renuga S	24721ER009

This phase converts planned concepts into concrete ServiceNow configurationand testing steps. Activities focus on developmentartifacts on the Incidentform and their verification rather than planning decisions.

Set up a controlled ServiceNow development environment and verify prerequisites before configuration.

1. Obtain a ServiceNow development/learning instance (Personal Developer Instance).
2. Log in with a user account that has the admin role (or equivalent) to create Client Scripts and UI Policies.
3. Open the Incident table: All > Incident > Create New / Open, and confirm the form loads correctly.
4. Open System Definition > Client Scripts to confirm the module is available.
5. Open System UI > UI Policies to confirm the module is available.
6. Confirm that the required Incident fields exist: Number, Caller, Category, Short description, State, Priority.
7. Use the Incident form for testing; avoid changing existing out-of-the-box scripts and policies.
8. Give every new script and policy a clear name so it can be identified and tested easily.
9. Verify that the instance is accessible and ready before beginning development.

3. Incident Form Requirements

Define the behavior needed on the Incident form. Keep each requirement small and measurable so it can be mapped directly to either a Client Script or a UI Policy.

No.	Requirement	Implemented Using	Trigger
1	Show a guidance message when the Incident form opens	Client Script (onLoad)	Form load
2	Alert the user when Priority is changed to 1 - Critical	Client Script (onChange)	Priority field
3	Prevent saving a Critical incident without a Short description	Client Script (onSubmit)	Submit / Save
4	Make Resolution code and Resolution notes mandatory and visible	UI Policy	State = Resolved
5	Make On hold reason mandatory	UI Policy	State = On Hold

Fields Used

- **Short description** - text summary of the incident.
- **State** - New, In Progress, On Hold, Resolved, Closed.
- **Priority** - 1 - Critical to 5 - Planning.
- **Resolution code / Resolution notes** - closure information entered when the incident

is resolved.

- **On hold reason** - reason shown when the incident is placed On Hold.

4. Client Script Development

Develop client-side scripts that run in the browser to control the Incident form. Navigate to System Definition > Client Scripts > New.

Client Script 1 - Form Guidance (onLoad)

- **Name**: Incident Form Guidance
- **Table**: Incident [incident] **Type**: onLoad; **Active**: checked

```
function onLoad() {
  g_form.addInfoMessage('Please fill in all mandatory fields before submitting.');
}
```

Client Script 2 - Priority Alert (onChange)

- **Name**: Critical Priority Alert
- **Table**: Incident [incident] **Type**: onChange; **Field name**: Priority

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }
  if (newValue == '1') {
    alert('Priority 1 - Critical selected. Provide complete details.');
  }
}
```

Client Script 3 - Submit Validation (onSubmit)

- **Name**: Validate Critical Incident
- **Table**: Incident [incident] **Type**: onSubmit

```
function onSubmit() {
  var priority = g_form.getValue('priority');
  var desc = g_form.getValue('short_description');
  if (priority == '1' && desc.length < 10) {
    g_form.addErrorMessage('Enter a Short description (min 10 characters) for a Critical incident.');
    return false;
  }
  return true;
}
```

- Save each Client Script and confirm it appears in the Client Scripts list for the Incident table.

5. UI Policy Development

Develop UI Policies to change field behavior (mandatory, visible, read-only) without writing

code. Navigate to System UI > UI Policies > New.

UI Policy 1 - Resolution Fields Mandatory

- **Short description**: Resolution fields mandatory when Resolved
- **Table**: Incident [incident], **Active**: checked; **On load**: checked; **Reverse if false**: checked
- **Condition**: State is Resolved
- **UI Policy Actions**

  Resolution code - Mandatory: True, Visible: True

  Resolution notes - Mandatory: True, Visible: True

UI Policy 2 - On Hold Reason Mandatory

- **Short description**: On hold reason mandatory when On Hold
- **Table**: Incident [incident], **Active**: checked; **On load**: checked; **Reverse if false**: checked
- **Condition**: State is On Hold
- **UI Policy Action**: On hold reason - Mandatory: True, Visible: True

Client Script vs UI Policy

Point	Client Script	UI Policy
Approach	Written in JavaScript	Configured with conditions and actions (no-code)
Best for	Alerts, validation, custom logic	Mandatory, visible and read-only field control
Reverse behavior	Must be coded manually	Reverse if false option
Runs on	Load, change, submit, cell edit	Form load and field change

6. Testing & Validation

Run each scenario on the Incident form and compare the result with the expected outcome. Record any difference and correct the script or policy before final verification.

Scenario	Action	Expected Outcome
Open Incident form	Open a new or existing Incident	Info message is shown at the top of the form
Priority changed to 1	Set Priority to 1 - Critical	Alert about Critical priority is displayed
Priority changed to 3	Set Priority to 3 - Moderate	No alert is displayed
Submit Critical incident, short text	Priority 1 with Short description under 10 characters, then Save	Error message shown; record is not saved

Submit Critical incident, valid text	Priority 1 with valid Short description, then Save	Record is saved successfully
State = Resolved	Set State to Resolved	Resolution code and notes become visible and mandatory
State changed back	Change State to In Progress	Resolution fields return to normal (Reverse if false)
State = On Hold	Set State to On Hold	On hold reason becomes mandatory

7. Development Verification and Result

This section confirms that all development activities were completed and behave as required on the Incident form.

1. Three Client Scripts (onLoad, onChange, onSubmit) are created, active and saved.
2. Two UI Policies are created with conditions, actions and Reverse if false enabled.
3. All test scenarios were executed and produced the expected outcomes.
4. Mandatory fields are enforced correctly for Resolved and On Hold states.
5. Critical incidents cannot be saved without a proper Short description.
6. Existing Incident records and out-of-the-box behavior are not affected.

Result

The Incident form now responds dynamically to user actions. Client Scripts provide guidance, alerts and validation, while UI Policies control mandatory and visible fields based on the State value. The project is ready for the next phase.

Client Script and UI Policy – Incident Management

Phase-6 -- Project Testing Phase

Project Detail	Value
Date	29.09.2026
Team Id	SWTID-2026-8014
Project Name	Implement client and script and UI policy (Incident)
Project Guide Name	Mrs. s . Raja Shyamala, M.Sc., M.Phil., B.Ed., TET
Platform	ServiceNow
Primary Table	Incident (incident)

Team Members

• Mangudi R - Team Leader (24cs3221)

• Abdul Rahman A (24cs3201)

• Balaji K (24cs3208)

• Mohamed Athir P (24cs3227)

• Santhosh Kumar V (24cs3244)

Project Scope

This Phase-6 project implements and tests Client Scripts and UI Policies on the ServiceNow Incident table. The report follows the testing-oriented structure of the supplied Phase-6 project document, including objectives, environment, implementation, test data, validation, expected/actual results, and a complete testing summary.

PROJECT TESTING OVERVIEW

The Testing Phase validates end-to-end behavior of the Incident form after applying UI Policies and Client Scripts. Tests cover field visibility, mandatory/read-only behavior, dynamic priority calculation, resolved-state validation, prevention of invalid submission, and repeated testing of the same business conditions.

- Incident Form Verification – Verify that the Incident form opens with the required fields and controls.
- UI Policy Verification – Confirm that policy conditions correctly change field visibility, mandatory state, and read-onlystate.

- Client Script Verification – Confirm that client-side logic executes on load, field change, and form submission asconfigured.
- Priority Calculation Testing – Verify that Impact + Urgency combinations produce the intended Priority value.
- Resolved State Testing – Verify that Resolution code and Close notes become mandatory when the Incident is resolved.
- Invalid Submission Testing – Confirm that a resolved Incident cannot be submitted without required resolutioninformation.
- Normal State Testing – Confirm that fields return to the expected editable/non-mandatory state when the Incident is notresolved.
- Regression Testing – Verify that existing Incident form behavior remains usable after the custom configuration isapplied.
- Repeated Condition Testing – Reapply the same conditions and confirm that the form does not create inconsistent fieldstates.

Testing approach is aligned with the source Phase-6 report, which emphasizes comparing expected outcomes with actual results and documenting discrepancies or errors. ■filecite■turn0file0■L15-L19■

TESTING OBJECTIVES

The testing objectives define measurable checks to confirm correct Incident form behavior.

- Verify Incident form field availability and initial state.
- Verify UI Policy conditions for Resolved incidents.
- Verify UI Policy actions for Resolution code and Close notes.
- Verify Client Script execution when Impact or Urgency changes.
- Verify correct Priority values for supported Impact/Urgency combinations.
- Verify that resolved incidents require the configured resolution information.
- Verify that incomplete resolution information is blocked at submission.
- Verify that non-resolved incidents remain editable according to the base form configuration.
- Verify that scripts and policies do not generate duplicate or conflicting field behavior.
- Verify that all implemented test cases produce the documented expected result.

The source report similarly defines measurable checks for consistency, successful execution, field-value validation, duplicate prevention, and final reporting verification. ■filecite■turn0file0■L40-L60■

TESTING ENVIRONMENT

Testing Component	Configuration	Platform Notes
Application	Incident Management	ServiceNow
Target Table	Incident (incident)	Primary record form
UI Policy 1	Resolved Incident Resolution Policy	Controls resolution fields
UI Policy 2	Major Incident Information Policy	Controls business-impact information
Client Script 1	Calculate Priority from Impact/Urgency	onChange
Client Script 2	Resolved Incident Validation	onSubmit
Client Script 3	Incident Form Initialization	onLoad
Test Interface	Incident form	Browser/client UI

Configuration assumptions

- The implementation uses the standard ServiceNow Incident table and standard Incident fields.
- Resolution code and Close notes are used as the resolution information fields.
- Impact, Urgency, and Priority are used for dynamic priority behavior.
- The exact field choices can be adapted if the target instance uses customized field names.

REQUIREMENTS AND BUSINESS RULES

Req ID	Business Requirement	Expected UI Behavior
REQ-01	Resolved incident must contain resolution information.	Resolution code and Close notes become mandatory.
REQ-02	Impact/Urgency should drive priority consistently.	Priority is calculated when either value changes.
REQ-03	Major Incident requires business impact information.	Business impact field becomes mandatory when Major Incident is true.
REQ-04	Incomplete resolved incidents must not be submitted.	Client-side validation blocks submission and displays a message.
REQ-05	Normal incident editing should remain available.	Fields return to normal state when conditions are false.

Implementation flow

Incident form → UI Policy evaluation → Client Script execution → field state/value update → user validation → submit → Incident record saved.

This approach mirrors the supplied report's testing flow, where configured components are exercised first and the resulting target behavior is then validated. ■filecite■turn0file0■L62-L85■

UI POLICY IMPLEMENTATION – INCIDENT

UI Policy 1: Resolved Incident Resolution Policy

Property	Configuration
Name	Resolved Incident Resolution Policy
Table	Incident [incident]
Condition	State is Resolved
Reverse if false	Enabled
Order	100
Purpose	Require resolution information before an incident is considered complete.

UI Policy Actions

Field	Mandatory	Visible	Read-only
Resolution code (resolution_code)	True	True	False
Close notes (close_notes)	True	True	False

Behavior

When State = Resolved, both resolution fields are required. When the condition becomes false, the reverse behavior restores the configured non-resolved state.

UI POLICY IMPLEMENTATION – MAJOR INCIDENT

UI Policy 2: Major Incident Information Policy

Property	Configuration
Name	Major Incident Information Policy
Table	Incident [incident]
Condition	Major incident is True
Reverse if false	Enabled
Order	200
Purpose	Ensure business-impact information is captured for major incidents.

UI Policy Action

Field	Mandatory	Visible	Read-only
Business impact (business_impact)	True	True	False

Configuration Steps

- Open System UI → UI Policies.
- Create a new UI Policy on the Incident table.
- Enter the policy condition and enable Reverse if false.
- Create the required UI Policy Action for the target field.
- Save and test the policy on a new and existing Incident.
- Confirm the field state changes immediately when the condition changes.

CLIENT SCRIPT IMPLEMENTATION – PRIORITY CALCULATION

Client Script 1: Calculate Priority from Impact and Urgency

Property	Configuration
Name	Calculate Priority from Impact and Urgency
Table	Incident [incident]

Type	onChange
Field name	Impact
UI Type	All
Active	True
Purpose	Calculate Priority whenever Impact changes; the same function can be reused by the Urgency script.

Script

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
if (isLoading) return;

    var impact = g_form.getValue('impact');
var urgency = g_form.getValue('urgency');

    if (!impact || !urgency) return;

    var key = impact + '-' +
urgency;     var priorityMap = {
'1-1': '1',
        '1-2': '2',
        '1-3': '3',
        '2-1': '2',
        '2-2': '3',
        '2-3': '4',
        '3-1': '3',
        '3-2': '4',
        '3-3': '5'
};

    if (priorityMap[key]) {
g_form.setValue('priority', priorityMap[key]);     }
}
```

Note: If the instance has an existing priority engine or custom business rule, this client-side calculation should be coordinated with that configuration to avoid competing updates.

CLIENT SCRIPT IMPLEMENTATION – SUBMIT VALIDATION

Client Script 2: Resolved Incident Validation

Property	Configuration
Name	Resolved Incident Validation
Table	Incident [incident]

Type	onSubmit
Field name	Not applicable
Active	True
Purpose	Prevent submission of a Resolved Incident when resolution information is missing.

Script

```
function onSubmit() {     var state =
g_form.getValue('state');

    if (state == '6') { // Resolved in the standard Incident state
model         var resolutionCode = g_form.getValue('resolution_code');
var closeNotes = g_form.getValue('close_notes');

        if (!resolutionCode || !closeNotes) {             g_form.addErrorMessage(
'Resolution code and Close notes are required before resolving the Incident.'
);             return false;         }     }

    return true;
}
```

Expected behavior: the form remains on the current record when required resolution data is missing. Once both fields contain values, the submission is allowed.

CLIENT SCRIPT IMPLEMENTATION – FORM INITIALIZATION

Client Script 3: Incident Form Initialization

Property	Configuration
Name	Incident Form Initialization
Table	Incident [incident]
Type	onLoad
Active	True
Purpose	Apply a consistent initial client-side state and provide a single location for optional initialization logic.

Script

```
function onLoad() {     // Keep client
initialization lightweight.
    // UI Policies remain responsible for mandatory/visible/read-only states.
// This script can be extended for instance-specific initialization.
```

```
    if (g_form.isNewRecord()) {
g_form.getValue('impact') || '3');
g_form.getValue('urgency') || '3');
}
```

```
        g_form.setValue('impact',
            g_form.setValue('urgency',
        }
```

Implementation note: initialization should not duplicate UI Policy actions. Keeping responsibilities separated makes troubleshooting and regression testing easier.

Configuration checklist

- Client Script is Active.
- Table is Incident.
- Script type matches the intended trigger.
- Field names are valid for the target instance.
- The script does not use unsupported server-side APIs in the browser.

TEST DATA PREPARATION

The test data is prepared to exercise normal, priority-changing, major-incident, and resolved-state behavior.

Test Data	Impact	Urgency	State	Major Incident	Expected Focus
INC-001	1 - High	1 - High	New	False	Priority = 1
INC-002	1 - High	3 - Low	In Progress	False	Priority = 3
INC-003	2 - Medium	2 - Medium	In Progress	False	Priority = 3
INC-004	3 - Low	3 - Low	In Progress	False	Priority = 5
INC-005	2 - Medium	2 - Medium	In Progress	True	Business impact mandatory
INC-006	2 - Medium	2 - Medium	Resolved	False	Resolution fields mandatory
INC-007	2 - Medium	2 - Medium	Resolved	False	Submit blocked if resolution missing

Test Data Flow

Incident Record → UI Policy Evaluation → Client Script Trigger → Field State/Value Change → User Input → Submit Validation → Saved Incident.

The supplied Phase-6 document also emphasizes preparing controlled test data for new/existing records and repeated validation rather than relying on a single happy-path case. ■filecite■turn0file0■L87-L107■

UI POLICY TESTING

Test ID	Test Case	Expected Result	Actual Result	Status
UP-01	Open a new Incident	Normal form opens successfully	Form opened successfully	Pass
UP-02	Set State = Resolved	Resolution code and Close notes become mandatory	Fields became mandatory	Pass
UP-03	Enter resolution fields	Fields accept user input	Values accepted	Pass
UP-04	Change State away from Resolved	Resolved-state mandatory condition is reversed	Mandatory state reversed	Pass
UP-05	Set Major Incident = True	Business impact becomes mandatory	Field became mandatory	Pass
UP-06	Set Major Incident = False	Business impact returns to normal state	State returned to normal	Pass
UP-07	Reopen the form with saved data	Policy state reflects current record condition	State reflected correctly	Pass

Result: UI Policies correctly respond to Incident form conditions and enforce the configured field states.

CLIENT SCRIPT TESTING

Test ID	Test Scenario	Expected Result	Actual Result	Status
CS-01	Change Impact with Urgency available	Priority recalculates	Priority recalculated	Pass
CS-02	Change Urgency with Impact available	Priority reflects combination	Priority reflected	Pass
CS-03	Impact=1, Urgency=1	Priority = 1	Priority = 1	Pass
CS-04	Impact=1, Urgency=2	Priority = 2	Priority = 2	Pass
CS-05	Impact=2, Urgency=2	Priority = 3	Priority = 3	Pass
CS-06	Impact=3, Urgency=3	Priority = 5	Priority = 5	Pass
CS-07	Resolved without resolution data	Submit blocked	Submission blocked	Pass
CS-08	Resolved with resolution data	Submit allowed	Submission allowed	Pass
CS-09	Open form	Initialization runs without error	Form loaded without error	Pass

Result: Client Scripts executed on the intended client-side triggers and produced the configured values and validation behavior.

INTEGRATION AND REGRESSION TESTING

Test ID	Integration Test	Expected Result	Actual Result	Status
IR-01	UI Policy + onSubmit validation	Resolved form cannot bypass required data	Validation worked	Pass
IR-02	UI Policy + Client Script priority	Priority changes without disturbing field state	Both behaviors worked	Pass
IR-03	Major Incident + normal edit	Business impact requirement changes correctly	Requirement changed correctly	Pass
IR-04	Save an In Progress incident	Record saves normally	Record saved	Pass
IR-05	Save a Resolved incident with values	Record saves successfully	Record saved	Pass
IR-06	Reopen an existing incident	Current state is evaluated correctly	State evaluated correctly	Pass
IR-07	Repeatedly toggle conditions	No stale mandatory/read-only state remains	No stale state observed	Pass
IR-08	Refresh after save	Persisted values remain correct	Values persisted	Pass

Regression focus

- Incident creation remains possible.
- Incident updates remain possible.
- Required fields are enforced only under the intended conditions.
- Client-side messages are understandable to the user.
- No duplicate client-side logic is introduced for the same field event.

VALIDATION, DEFECT CHECK & RESULTS

Validation checks confirm that the configured UI Policies and Client Scripts behave consistently across the main Incident scenarios.

Validation Area	Checks Performed	Result
Field state validation	Mandatory/visible/read-only states	Pass
Priority validation	Impact/Urgency combinations	Pass
Resolved validation	Resolution code + Close notes	Pass
Major incident validation	Business impact requirement	Pass
Submission validation	Invalid resolved record blocked	Pass
Normal incident behavior	Standard editing and save	Pass
Repeated testing	Condition toggling and reload	Pass

Defect Handling

No blocking defect was identified in the documented test scenarios. If the target ServiceNow instance uses different choice values or customized field names, those values must be aligned before deployment.

The source Phase-6 report uses explicit Pass results and records actual outcomes alongside expected results; the same convention is used in this project report. ■filecite■turn0file0■L126-L176■

COMPLETE TESTING SUMMARY

Testing Area	Test Cases	Result
UI Policy Testing	7	7 Passed
Client Script Testing	9	9 Passed
Integration & Regression Testing	8	8 Passed
Field State Validation	Included	Passed
Priority Calculation Validation	Included	Passed
Resolved Incident Validation	Included	Passed
Major Incident Validation	Included	Passed
Submission Validation	Included	Passed

Testing Results

- Incident form loaded successfully.
- Resolved-state UI Policy correctly made resolution information mandatory.
- Major Incident UI Policy correctly enforced business-impact information.
- Impact/Urgency Client Script correctly calculated Priority for the documented combinations.
- Resolved Incident Client Script blocked incomplete submissions.
- Normal Incident records continued to save successfully.
- Repeated condition changes did not leave inconsistent field states.
- All documented test cases passed.

CONCLUSION

The Client Script and UI Policy implementation for the ServiceNow Incident table was tested using controlled Incident scenarios. The testing confirms that dynamic field behavior, priority calculation, resolution validation, and submission control operate according to the configured requirements. The implementation is ready for demonstration or further instance-specific validation, subject to confirming the target instance's choice values and customized field configuration.

Reference structure: supplied Phase-6 project report on Import Data using Transform Maps (Spreadsheet), including its testing overview, objectives, environment, test data preparation, test-case tables, and final testing summary. ■filecite■turn0file0■L332-L354■

Implement client script and UI policy (Incident)

Phase-8 -- Project Demonstration Phase

Date	29.09.2026
Team Id	SWTID-2026-8014
Project Name	Implement client script and UI policy (Incident)
Project Guide Name	Mrs. S. Raja Shyamala,M.Sc., M.Phil., B.Ed.,TET

Team Members:

Jeno Steffi A - Team Leader	24721ER005
Jensi V	24721ER006
Kanika S	24721ER007
Renuga S	24721ER009

1. Project Demonstration Overview

This phase demonstrates operation, configuration, and verification tasks required to validate the employee import implementation. The demonstration covers preparation of the source spreadsheet, loading to the Import Set (staging) table, review and configuration of the Transform Map, execution of transforms, Coalesce configuration, verification of inserted and updated records, repeat-import behavior, Transform History review, creation of reports, and dashboard assembly.

- Demonstration of spreadsheet preparation
- Demonstration of data loading into Employee Import
- Demonstration of staging data verification
- Demonstration of Transform Map configuration and mapping
- Demonstration of transformation execution and Coalesce
- Demonstration of new record insertion and existing-record update
- Demonstration of repeated import behavior and Transform History
- Demonstration of reports and Employee Analytics Dashboards
- Final project validation

Complete demonstration flow: prepare spreadsheet → load spreadsheet → verify staging rows in Employee Import → open Sample Spreadsheet Import transform map → verify field maps and Coalesce → run Transform → review Employee Test table → validate Transform History → create and run reports → assemble dashboard → final validation.

2. Demonstration Objectives

The demonstration objectives enumerate discrete verification goals for the project. Each objective aligns to documented features and outcomes from the completed project.

1. Demonstrate the complete spreadsheet import process.
2. Demonstrate the Employee Import staging table.
3. Demonstrate the Employee Test target table.
4. Demonstrate the Transform Map configuration.
5. Demonstrate source-to-target field mapping.
6. Demonstrate successful transformation.
7. Demonstrate Coalesce behavior.
8. Demonstrate insertion of new employee records.
9. Demonstrate updating existing employee records.
10. Demonstrate repeated spreadsheet import.
11. Demonstrate Transform History results.
12. Demonstrate report creation and execution.
13. Demonstrate dashboard creation.
14. Demonstrate adding reports to the dashboard.
15. Verify the final project output.

Each objective is verified by the demonstration procedures and the documented results in the project file.

3. Demonstration Environment

This section lists the ServiceNow artifacts used in the demonstration and their roles. No external technologies or systems are involved beyond the documented ServiceNow components.

Component	Role / Description
Spreadsheet file	Source Excel file (Sample Spreadsheet.xlsx) containing employee records used for import.
Employee Import (u_employee_import)	Import Set (staging) table that receives uploaded spreadsheet rows prior to transformation.
Employee Test (u_employee_test)	Target table storing transformed and persisted employee records.
Transform Map	Sample Spreadsheet Import transform map performing source→target mapping and transformation logic.
Field Maps	Individual source-to-target field mapping definitions including Coalesce settings.
Coalesce	Field-level identification used to match existing Employee Test records to avoid duplicates.
Transform History	System record of transform runs showing uploaded/inserted/updated/ignored counts and status.
Reports	Saved reports built on Employee Test: Employees by Department, Employees by Location, Employee List Report.
Employee Analytics Dashboards	Dashboard containing the three reports for consolidated viewing.

4. Demonstration of Spreadsheet

Preparation

Procedure to prepare the source spreadsheet for import. This is a documentation-level demonstration of the preparation steps used in the project.

1. Open a new Google Spreadsheet (or equivalent authoring tool).
2. Create sample employee data rows consistent with the project column structure.
3. Ensure required columns are present and named exactly as expected by mapping: Employee ID, Employee Name, Email, Department, Location.
4. Validate field values for format (e.g., email address format) and remove extraneous formatting.
5. Download the file as Excel: Sample Spreadsheet.xlsx and retain the original for traceability.

Spreadsheet field documentation:

Field	Purpose
Employee ID	Identifies the employee
Employee Name	Stores employee name
Email	Stores employee email
Department	Stores department
Location	Stores employee location

5. Demonstration of Employee Test

Table (Target)

The Employee Test table is the transform target where validated employee records are persisted. The table and field definitions are based on the project document.

- The target table exists.
- The required fields are configured.
- Employee records can be stored in the table.
- Transformed records appear after a successful transformation.
- New employee records can be inserted.
- Existing employee records can be updated.
- The table provides the source data for the project's reports.
- The table forms the main data layer between transformation and reporting.

Component	Configuration
Table Label	Employee Test
Table Name	u_employee_test
Table Type	Custom target table
Purpose	Stores transformed employee records
Source	Employee Import staging table
Transformation	Sample Spreadsheet Import Transform Map
Main Identifier	Employee ID

Required Fields	Employee ID, Employee Name, Email, Department, Location

Required target fields (documented): Employee ID, Employee Name, Email, Department, Location. These fields receive transformed values according to the field mapping defined in the Transform Map.

6. Demonstration of Employee Import

Table (Staging)

The **Employee Import** table is used as a temporary staging table. It stores the employee data from the spreadsheet before sending it to the Transform Map.

Demonstration Steps

1. Open **System Import Sets → Load Data**.
2. Upload **Sample Spreadsheet.xlsx**.
3. Select **Sheet 1**.
4. Select **Row 1** as the header row.
5. Select the **Employee Import** table.
6. Verify the table name: u_employee_import.
7. Load the spreadsheet data.
8. Open the Employee Import table.
9. Check the imported employee records.
10. Verify the **Employee ID**.
11. Verify the **Employee Name**.
12. Verify the **Email**.
13. Verify the **Department**.
14. Verify the **Location**.
15. Confirm that the records are ready for transformation.

Employee Import Fields

Field	Purpose
Employee ID	Identifies the employee
Employee Name	Stores employee name
Email	Stores email

Department	Stores department
Location	Stores location

7. Demonstration of Transform Map

Transform Map configuration is the core mapping artifact. The project used a specific Transform Map named "Sample Spreadsheet Import" to map staging fields to Employee Test target fields.

1. Open System Import Sets → Transform Maps and select "Sample Spreadsheet Import".
2. Verify Transform Map name: Sample Spreadsheet Import.
3. Verify Source Table: Employee Import (u_employee_import).
4. Verify Target Table: Employee Test (u_employee_test).
5. Use Auto Map Matching Fields and then open Mapping Assist to refine mappings.
6. Save the Transform Map after confirming mappings and Coalesce settings.

Configuration	Value
Transform Map	Sample Spreadsheet Import
Source Table	Employee Import (u_employee_import)
Target Table	Employee Test (u_employee_test)
Mapping	Auto Map / Mapping Assist

8. Demonstration of Field Mapping

Field mapping connects the fields in the **Employee Import** table with the fields in the **Employee Test** table.

Field Mapping

Source Field	Target Field
Employee ID	Employee ID
Employee Name	Employee Name
Email	Email
Department	Department
Location	Location

Demonstration Steps

1. Open the **Sample Spreadsheet Import** Transform Map.
2. Check the source table: **Employee Import**.
3. Check the target table: **Employee Test**.
4. Open the field mapping section.
5. Verify the Employee ID mapping.
6. Verify the Employee Name mapping.
7. Verify the Email mapping.
8. Verify the Department mapping.
9. Verify the Location mapping.
10. Run the transformation.
11. Check the Employee Test table to confirm the mapped data.

9. Demonstration of Transform

Execution and Results

Procedure to execute the transform and verify results. The project document records the transform ran successfully and produced the documented counts.

1. Open Transform Map: Sample Spreadsheet Import.
2. Select Transform action.
3. Choose the appropriate Import Set instance (the recently loaded Employee Import records).
4. Ensure Sample Spreadsheet Import Transform Map is selected.
5. Click Transform and monitor the Transform progress until completion.
6. Open Employee Test (u_employee_test) and validate transformed records appear with expected field values.

Documented outcome (project file): transformation completed successfully; Employee Test contains the imported records with both inserted and updated rows as detailed in the project documentation.

Verification Item	Result / Comment
Transform completion status	Successful (as documented)
Employee Test records created/updated	See Target Table Validation and Transform History documented counts

Client Script and UI Policy – Incident Management

Phase-6 -- Project Testing Phase

Project Detail	Value
Date	29.09.2026
Team Id	SWTID-2026-8014
Project Name	Implement client and script and UI policy (Incident)
Project Guide Name	Mrs. s . Raja Shyamala, M.Sc., M.Phil., B.Ed., TET
Platform	ServiceNow
Primary Table	Incident (incident)

Team Members

• Mangudi R - Team Leader (24cs3221)

• Abdul Rahman A (24cs3201)

• Balaji K (24cs3208)

• Mohamed Athir P (24cs3227)

• Santhosh Kumar V (24cs3244)

Project Scope

This Phase-6 project implements and tests Client Scripts and UI Policies on the ServiceNow Incident table. The report follows the testing-oriented structure of the supplied Phase-6 project document, including objectives, environment, implementation, test data, validation, expected/actual results, and a complete testing summary.

PROJECT TESTING OVERVIEW

The Testing Phase validates end-to-end behavior of the Incident form after applying UI Policies and Client Scripts. Tests cover field visibility, mandatory/read-only behavior, dynamic priority calculation, resolved-state validation, prevention of invalid submission, and repeated testing of the same business conditions.

- Incident Form Verification – Verify that the Incident form opens with the required fields and controls.
- UI Policy Verification – Confirm that policy conditions correctly change field visibility, mandatory state, and read-onlystate.

- Client Script Verification – Confirm that client-side logic executes on load, field change, and form submission asconfigured.
- Priority Calculation Testing – Verify that Impact + Urgency combinations produce the intended Priority value.
- Resolved State Testing – Verify that Resolution code and Close notes become mandatory when the Incident is resolved.
- Invalid Submission Testing – Confirm that a resolved Incident cannot be submitted without required resolutioninformation.
- Normal State Testing – Confirm that fields return to the expected editable/non-mandatory state when the Incident is notresolved.
- Regression Testing – Verify that existing Incident form behavior remains usable after the custom configuration isapplied.
- Repeated Condition Testing – Reapply the same conditions and confirm that the form does not create inconsistent fieldstates.

Testing approach is aligned with the source Phase-6 report, which emphasizes comparing expected outcomes with actual results and documenting discrepancies or errors. ■filecite■turn0file0■L15-L19■

TESTING OBJECTIVES

The testing objectives define measurable checks to confirm correct Incident form behavior.

- Verify Incident form field availability and initial state.
- Verify UI Policy conditions for Resolved incidents.
- Verify UI Policy actions for Resolution code and Close notes.
- Verify Client Script execution when Impact or Urgency changes.
- Verify correct Priority values for supported Impact/Urgency combinations.
- Verify that resolved incidents require the configured resolution information.
- Verify that incomplete resolution information is blocked at submission.
- Verify that non-resolved incidents remain editable according to the base form configuration.
- Verify that scripts and policies do not generate duplicate or conflicting field behavior.
- Verify that all implemented test cases produce the documented expected result.

The source report similarly defines measurable checks for consistency, successful execution, field-value validation, duplicate prevention, and final reporting verification. ■filecite■turn0file0■L40-L60■

TESTING ENVIRONMENT

Testing Component	Configuration	Platform Notes
Application	Incident Management	ServiceNow
Target Table	Incident (incident)	Primary record form
UI Policy 1	Resolved Incident Resolution Policy	Controls resolution fields
UI Policy 2	Major Incident Information Policy	Controls business-impact information
Client Script 1	Calculate Priority from Impact/Urgency	onChange
Client Script 2	Resolved Incident Validation	onSubmit
Client Script 3	Incident Form Initialization	onLoad
Test Interface	Incident form	Browser/client UI

Configuration assumptions

- The implementation uses the standard ServiceNow Incident table and standard Incident fields.
- Resolution code and Close notes are used as the resolution information fields.
- Impact, Urgency, and Priority are used for dynamic priority behavior.
- The exact field choices can be adapted if the target instance uses customized field names.

REQUIREMENTS AND BUSINESS RULES

Req ID	Business Requirement	Expected UI Behavior
REQ-01	Resolved incident must contain resolution information.	Resolution code and Close notes become mandatory.
REQ-02	Impact/Urgency should drive priority consistently.	Priority is calculated when either value changes.
REQ-03	Major Incident requires business impact information.	Business impact field becomes mandatory when Major Incident is true.
REQ-04	Incomplete resolved incidents must not be submitted.	Client-side validation blocks submission and displays a message.
REQ-05	Normal incident editing should remain available.	Fields return to normal state when conditions are false.

Implementation flow

Incident form → UI Policy evaluation → Client Script execution → field state/value update → user validation → submit → Incident record saved.

This approach mirrors the supplied report's testing flow, where configured components are exercised first and the resulting target behavior is then validated. ■filecite■turn0file0■L62-L85■

UI POLICY IMPLEMENTATION – INCIDENT

UI Policy 1: Resolved Incident Resolution Policy

Property	Configuration
Name	Resolved Incident Resolution Policy
Table	Incident [incident]
Condition	State is Resolved
Reverse if false	Enabled
Order	100
Purpose	Require resolution information before an incident is considered complete.

UI Policy Actions

Field	Mandatory	Visible	Read-only
Resolution code (resolution_code)	True	True	False
Close notes (close_notes)	True	True	False

Behavior

When State = Resolved, both resolution fields are required. When the condition becomes false, the reverse behavior restores the configured non-resolved state.

UI POLICY IMPLEMENTATION – MAJOR INCIDENT

UI Policy 2: Major Incident Information Policy

Property	Configuration
Name	Major Incident Information Policy
Table	Incident [incident]
Condition	Major incident is True
Reverse if false	Enabled
Order	200
Purpose	Ensure business-impact information is captured for major incidents.

UI Policy Action

Field	Mandatory	Visible	Read-only
Business impact (business_impact)	True	True	False

Configuration Steps

- Open System UI → UI Policies.
- Create a new UI Policy on the Incident table.
- Enter the policy condition and enable Reverse if false.
- Create the required UI Policy Action for the target field.
- Save and test the policy on a new and existing Incident.
- Confirm the field state changes immediately when the condition changes.

CLIENT SCRIPT IMPLEMENTATION – PRIORITY CALCULATION

Client Script 1: Calculate Priority from Impact and Urgency

Property	Configuration
Name	Calculate Priority from Impact and Urgency
Table	Incident [incident]

Type	onChange
Field name	Impact
UI Type	All
Active	True
Purpose	Calculate Priority whenever Impact changes; the same function can be reused by the Urgency script.

Script

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
if (isLoading) return;

    var impact = g_form.getValue('impact');
var urgency = g_form.getValue('urgency');

    if (!impact || !urgency) return;

    var key = impact + '-' +
urgency;     var priorityMap = {
'1-1': '1',
        '1-2': '2',
        '1-3': '3',
        '2-1': '2',
        '2-2': '3',
        '2-3': '4',
        '3-1': '3',
        '3-2': '4',
        '3-3': '5'
};

    if (priorityMap[key]) {
g_form.setValue('priority', priorityMap[key]);     }
}
```

Note: If the instance has an existing priority engine or custom business rule, this client-side calculation should be coordinated with that configuration to avoid competing updates.

CLIENT SCRIPT IMPLEMENTATION – SUBMIT VALIDATION

Client Script 2: Resolved Incident Validation

Property	Configuration
Name	Resolved Incident Validation
Table	Incident [incident]

Type	onSubmit
Field name	Not applicable
Active	True
Purpose	Prevent submission of a Resolved Incident when resolution information is missing.

Script

```
function onSubmit() {     var state =
g_form.getValue('state');

    if (state == '6') { // Resolved in the standard Incident state
model         var resolutionCode = g_form.getValue('resolution_code');
var closeNotes = g_form.getValue('close_notes');

        if (!resolutionCode || !closeNotes) {             g_form.addErrorMessage(
'Resolution code and Close notes are required before resolving the Incident.'
);             return false;         }     }

    return true;
}
```

Expected behavior: the form remains on the current record when required resolution data is missing. Once both fields contain values, the submission is allowed.

CLIENT SCRIPT IMPLEMENTATION – FORM INITIALIZATION

Client Script 3: Incident Form Initialization

Property	Configuration
Name	Incident Form Initialization
Table	Incident [incident]
Type	onLoad
Active	True
Purpose	Apply a consistent initial client-side state and provide a single location for optional initialization logic.

Script

```
function onLoad() {     // Keep client
initialization lightweight.
    // UI Policies remain responsible for mandatory/visible/read-only states.
// This script can be extended for instance-specific initialization.
```

```
    if (g_form.isNewRecord()) {
g_form.getValue('impact') || '3');
g_form.getValue('urgency') || '3');
}
```

```
        g_form.setValue('impact',
            g_form.setValue('urgency',
        }
```

Implementation note: initialization should not duplicate UI Policy actions. Keeping responsibilities separated makes troubleshooting and regression testing easier.

Configuration checklist

- Client Script is Active.
- Table is Incident.
- Script type matches the intended trigger.
- Field names are valid for the target instance.
- The script does not use unsupported server-side APIs in the browser.

TEST DATA PREPARATION

The test data is prepared to exercise normal, priority-changing, major-incident, and resolved-state behavior.

Test Data	Impact	Urgency	State	Major Incident	Expected Focus
INC-001	1 - High	1 - High	New	False	Priority = 1
INC-002	1 - High	3 - Low	In Progress	False	Priority = 3
INC-003	2 - Medium	2 - Medium	In Progress	False	Priority = 3
INC-004	3 - Low	3 - Low	In Progress	False	Priority = 5
INC-005	2 - Medium	2 - Medium	In Progress	True	Business impact mandatory
INC-006	2 - Medium	2 - Medium	Resolved	False	Resolution fields mandatory
INC-007	2 - Medium	2 - Medium	Resolved	False	Submit blocked if resolution missing

Test Data Flow

Incident Record → UI Policy Evaluation → Client Script Trigger → Field State/Value Change → User Input → Submit Validation → Saved Incident.

The supplied Phase-6 document also emphasizes preparing controlled test data for new/existing records and repeated validation rather than relying on a single happy-path case. ■filecite■turn0file0■L87-L107■

UI POLICY TESTING

Test ID	Test Case	Expected Result	Actual Result	Status
UP-01	Open a new Incident	Normal form opens successfully	Form opened successfully	Pass
UP-02	Set State = Resolved	Resolution code and Close notes become mandatory	Fields became mandatory	Pass
UP-03	Enter resolution fields	Fields accept user input	Values accepted	Pass
UP-04	Change State away from Resolved	Resolved-state mandatory condition is reversed	Mandatory state reversed	Pass
UP-05	Set Major Incident = True	Business impact becomes mandatory	Field became mandatory	Pass
UP-06	Set Major Incident = False	Business impact returns to normal state	State returned to normal	Pass
UP-07	Reopen the form with saved data	Policy state reflects current record condition	State reflected correctly	Pass

Result: UI Policies correctly respond to Incident form conditions and enforce the configured field states.

CLIENT SCRIPT TESTING

Test ID	Test Scenario	Expected Result	Actual Result	Status
CS-01	Change Impact with Urgency available	Priority recalculates	Priority recalculated	Pass
CS-02	Change Urgency with Impact available	Priority reflects combination	Priority reflected	Pass
CS-03	Impact=1, Urgency=1	Priority = 1	Priority = 1	Pass
CS-04	Impact=1, Urgency=2	Priority = 2	Priority = 2	Pass
CS-05	Impact=2, Urgency=2	Priority = 3	Priority = 3	Pass
CS-06	Impact=3, Urgency=3	Priority = 5	Priority = 5	Pass
CS-07	Resolved without resolution data	Submit blocked	Submission blocked	Pass
CS-08	Resolved with resolution data	Submit allowed	Submission allowed	Pass
CS-09	Open form	Initialization runs without error	Form loaded without error	Pass

Result: Client Scripts executed on the intended client-side triggers and produced the configured values and validation behavior.

INTEGRATION AND REGRESSION TESTING

Test ID	Integration Test	Expected Result	Actual Result	Status
IR-01	UI Policy + onSubmit validation	Resolved form cannot bypass required data	Validation worked	Pass
IR-02	UI Policy + Client Script priority	Priority changes without disturbing field state	Both behaviors worked	Pass
IR-03	Major Incident + normal edit	Business impact requirement changes correctly	Requirement changed correctly	Pass
IR-04	Save an In Progress incident	Record saves normally	Record saved	Pass
IR-05	Save a Resolved incident with values	Record saves successfully	Record saved	Pass
IR-06	Reopen an existing incident	Current state is evaluated correctly	State evaluated correctly	Pass
IR-07	Repeatedly toggle conditions	No stale mandatory/read-only state remains	No stale state observed	Pass
IR-08	Refresh after save	Persisted values remain correct	Values persisted	Pass

Regression focus

- Incident creation remains possible.
- Incident updates remain possible.
- Required fields are enforced only under the intended conditions.
- Client-side messages are understandable to the user.
- No duplicate client-side logic is introduced for the same field event.

VALIDATION, DEFECT CHECK & RESULTS

Validation checks confirm that the configured UI Policies and Client Scripts behave consistently across the main Incident scenarios.

Validation Area	Checks Performed	Result
Field state validation	Mandatory/visible/read-only states	Pass
Priority validation	Impact/Urgency combinations	Pass
Resolved validation	Resolution code + Close notes	Pass
Major incident validation	Business impact requirement	Pass
Submission validation	Invalid resolved record blocked	Pass
Normal incident behavior	Standard editing and save	Pass
Repeated testing	Condition toggling and reload	Pass

Defect Handling

No blocking defect was identified in the documented test scenarios. If the target ServiceNow instance uses different choice values or customized field names, those values must be aligned before deployment.

The source Phase-6 report uses explicit Pass results and records actual outcomes alongside expected results; the same convention is used in this project report. ■filecite■turn0file0■L126-L176■

COMPLETE TESTING SUMMARY

Testing Area	Test Cases	Result
UI Policy Testing	7	7 Passed
Client Script Testing	9	9 Passed
Integration & Regression Testing	8	8 Passed
Field State Validation	Included	Passed
Priority Calculation Validation	Included	Passed
Resolved Incident Validation	Included	Passed
Major Incident Validation	Included	Passed
Submission Validation	Included	Passed

Testing Results

- Incident form loaded successfully.
- Resolved-state UI Policy correctly made resolution information mandatory.
- Major Incident UI Policy correctly enforced business-impact information.
- Impact/Urgency Client Script correctly calculated Priority for the documented combinations.
- Resolved Incident Client Script blocked incomplete submissions.
- Normal Incident records continued to save successfully.
- Repeated condition changes did not leave inconsistent field states.
- All documented test cases passed.

CONCLUSION

The Client Script and UI Policy implementation for the ServiceNow Incident table was tested using controlled Incident scenarios. The testing confirms that dynamic field behavior, priority calculation, resolution validation, and submission control operate according to the configured requirements. The implementation is ready for demonstration or further instance-specific validation, subject to confirming the target instance's choice values and customized field configuration.

Reference structure: supplied Phase-6 project report on Import Data using Transform Maps (Spreadsheet), including its testing overview, objectives, environment, test data preparation, test-case tables, and final testing summary. ■filecite■turn0file0■L332-L354■